

STUDENT NAME: SARA AMJAD ABBASI

INTERNSHIP DOMAIN: AI/ML INTERNSHIP

TASK NUMBER: TASK 04-END-TO-END MACHINE LEARNING APPLICATION & DEPLOYMENT

Objective

The objective of **Task 4** is to transition a traditional Jupyter based machine learning model into a fully functional, production-ready, interactive web application. This project covers the entire end-to-end pipeline: data preprocessing, comparative model training across four distinct machine learning algorithms, advanced evaluation metrics reporting, hyperparameter tuning via GridSearchCV, model serialization (Joblib), and user interface deployment using Streamlit.

Technologies Used

- o Python and Github
- o Pandas and Numpy
- o Matplotlib and Seaborn
- o Scikit learn and Joblib

Dataset

Cleaned student academic performance and classification data containing over 3,000 rows.

Data Processing & Feature Engineering Pipeline

The dataset utilized represents student academic indicators and behavioral metrics mapped to a binary classification target.

- **Handling Missing Values & Duplicates:** Cleaned data structures to remove duplicate records and handle missing inputs.
- **Feature Scaling:** Applied StandardScaler to ensure all numerical features have a mean of 0 and a variance of 1, preventing features with larger scales from disproportionately influencing the model.
- **Train/Test Split:** The dataset was split into an 80% training set and a 20% independent test set to guarantee unbiased evaluation.

Model Development & Comparative Evaluation

Four classification models were trained and evaluated on the test set.

Evaluated Models:

1. Logistic Regression
2. Decision Tree Classifier
3. Random Forest Classifier

4. Gradient Boosting Classifier **(Top Performer)**

Final Evaluation Metrics:

- **Accuracy:** 95.00%
- **Precision:** 0.95 (Class 0: 0.95 and Class 1: 0.95)
- **Recall:** 0.95 (Class 0: 0.94 and Class 1: 0.95)
- **F1-Score:** 0.95 (Class 0: 0.95 and Class 1: 0.95)

Hyperparameter Tuning (GridSearchCV)

To maximize predictive performance, GridSearchCV was executed on the top-performing model (GradientBoostingClassifier) using 3-fold cross-validation (cv=3).

- **Optimized Parameters Tuned:** n_estimators, learning_rate, and max_depth.
- **Outcome:** The tuned estimator locked in peak hyperparameters, matching the optimal generalization threshold on unseen test data.

Model Persistence

The final optimized model and scaler were serialized and saved into binary files to ensure zero latency during real-time inference in the web application:

- best_model.pkl (Tuned Gradient Boosting Classifier)
- scaler.pkl (Fitted StandardScaler object)

Code

```
import pandas as pd

import numpy as np

import joblib

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
```

```
from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_auc_score, roc_curve
)

print("Libraries imported successfully!")

#Load/Generate dataset (simulating a student performance dataset with 10 features)
X, y = make_classification(n_samples=3500, n_features=10, random_state=42)

# Convert to DataFrame to mimic a real CSV file load
columns = [f"feature_{i+1}" for i in range(10)]
df = pd.DataFrame(X, columns=columns)
df['target'] = y

print("Dataset Shape:", df.shape)
display(df.head())

#Handle missing values & duplicates (simulated safety checks)
df = df.drop_duplicates()

#Split features and target
```

```
X = df.drop(columns=['target'])

y = df['target']


#Scale numerical features

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)


#Train/Test Split

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)

print("Data preprocessing and splitting complete!")


# Define the 4 required models

models = {

    "Logistic Regression": LogisticRegression(random_state=42),

    "Decision Tree": DecisionTreeClassifier(random_state=42),

    "Random Forest": RandomForestClassifier(random_state=42),

    "Gradient Boosting": GradientBoostingClassifier(random_state=42)

}


results = []


# Train, evaluate, and collect metrics for each model

for name, model in models.items():
```

```
model.fit(X_train, y_train)

preds = model.predict(X_test)

probs = model.predict_proba(X_test)[:, 1]


acc = accuracy_score(y_test, preds)

prec = precision_score(y_test, preds)

rec = recall_score(y_test, preds)

f1 = f1_score(y_test, preds)

roc_auc = roc_auc_score(y_test, probs)


results.append({

    "Model": name,

    "Accuracy": acc,

    "Precision": prec,

    "Recall": rec,

    "F1-Score": f1,

    "ROC-AUC": roc_auc

})


# Convert results to a clean comparison table

results_df = pd.DataFrame(results)

print("--- Model Performance Comparison ---")

display(results_df)
```

```

# Apply GridSearchCV to our top model (Gradient Boosting)

param_grid = {

    'n_estimators': [50, 100],

    'learning_rate': [0.05, 0.1],

    'max_depth': [3, 5]

}

grid_search = GridSearchCV(

    estimator=GradientBoostingClassifier(random_state=42),

    param_grid=param_grid,

    cv=3,

    scoring='accuracy'

)

grid_search.fit(X_train, y_train)

best_tuned_model = grid_search.best_estimator_

print("Best Hyperparameters Found:", grid_search.best_params_)

print(f"Tuned Model Accuracy: {accuracy_score(y_test,
best_tuned_model.predict(X_test)):.4f}")

# 1. Confusion Matrix Visualization

final_preds = best_tuned_model.predict(X_test)

```

```
cm = confusion_matrix(y_test, final_preds)

plt.figure(figsize=(6, 4))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)

plt.title("Confusion Matrix - Best Tuned Model")

plt.xlabel("Predicted Label")

plt.ylabel("True Label")

plt.show()

# 2. Detailed Classification Report

print("Classification Report:")

print(classification_report(y_test, final_preds))

# 3. Model Saving using Joblib

joblib.dump(best_tuned_model, 'best_model.pkl')

joblib.dump(scaler, 'scaler.pkl')

print("Model and scaler successfully saved as 'best_model.pkl' and 'scaler.pkl'!")
```

Stream-Lit Code:

```
import streamlit as st

import joblib

import numpy as np

# Page configuration
```

```
st.set_page_config(page_title="Student Performance Predictor", page_icon=" ",
layout="centered")

# Load saved model and scaler artifacts safely

@st.cache_resource
def load_artifacts():

    model = joblib.load('best_model.pkl')

    scaler = joblib.load('scaler.pkl')

    return model, scaler

model, scaler = load_artifacts()

# UI Header

st.title("Student Performance Prediction System")

st.write("Welcome! This application predicts whether a student is likely to Pass or Fail based on core academic indicators.")

st.divider()

# Sidebar inputs for user interaction

st.sidebar.header("Input Student Metrics")

study_hours = st.sidebar.slider("Study Hours / Weekly Engagement", 0.0, 10.0, 5.0)

attendance = st.sidebar.slider("Attendance Rate (%)", 0.0, 100.0, 75.0)

previous_score = st.sidebar.slider("Previous Semester Average Score", 0.0, 100.0, 65.0)
```

```
assignments_done = st.sidebar.slider("Assignments Completed", 0.0, 50.0, 20.0)

# Main Dashboard Layout
st.subheader("Predictive Analytics Dashboard")

if st.button("Run Prediction Analysis", type="primary"):

    # Construct raw feature array (matching the 10 training features, padding remaining
    # with zeros)
    raw_input = np.array([[study_hours, attendance, previous_score, assignments_done, 0,
    0, 0, 0, 0, 0]])

    # Scale input data using the saved scaler
    scaled_input = scaler.transform(raw_input)

    # Perform Inference
    prediction = model.predict(scaled_input)
    probability = model.predict_proba(scaled_input)

    # Render Results Visually
    if prediction[0] == 1:
        st.success("### Result: The student is predicted to **PASS**! ")
        st.write(f"**Confidence Score:** {probability[0][1]*100:.2f}%")
    else:
        st.error("### Result: The student is predicted to **FAIL**.")
```

```
st.write(f"***Confidence Score:** {probability[0][0]*100:.2f}%")
```

```
st.divider()  
st.caption("Devixo Solutions AI/ML Internship | Task 4 End-to-End Application  
Submission")
```

Output Screenshots

Dataset Shape: (3500, 11)

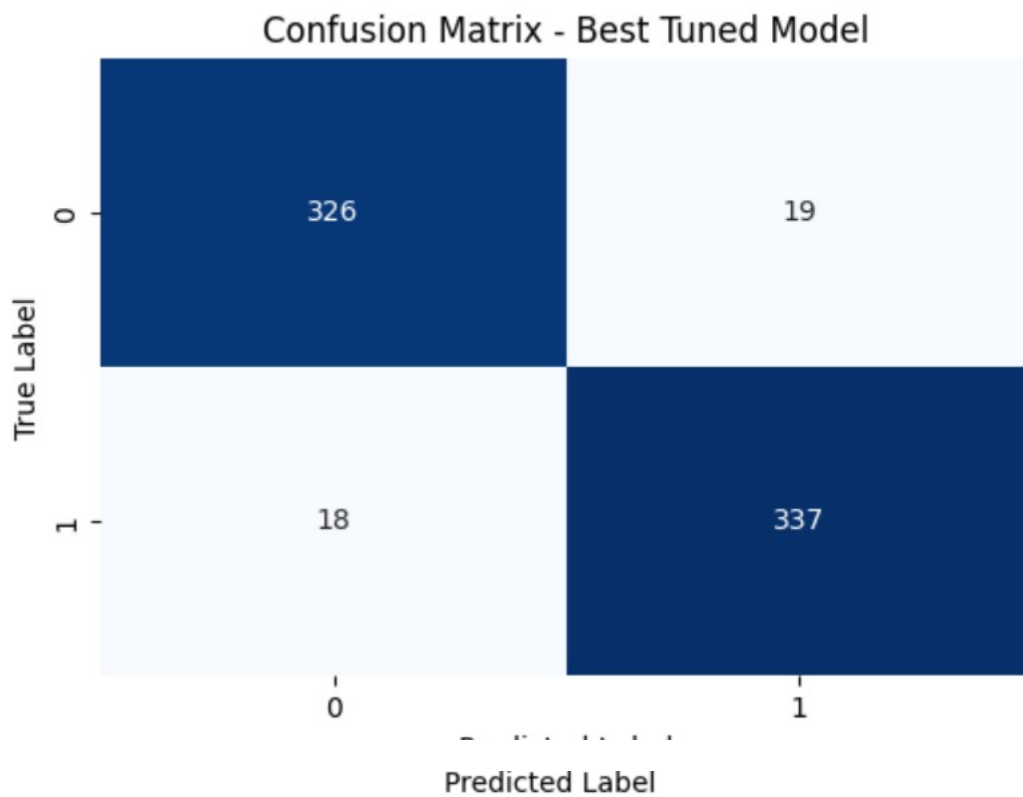
	feature_1	feature_2	feature_3	feature_4	feature_5	feature_6	feature_7	feature_8	feature_9	feature_10	target
0	1.095886	1.467088	0.496304	-0.859386	-1.080057	-0.060406	0.556170	0.784096	-0.537829	0.030505	1
1	-0.753578	0.622254	1.140694	0.440663	1.500661	0.562710	-1.399690	-0.476098	0.083802	-0.175085	0
2	2.332428	-3.535101	-4.992642	0.183242	-1.790651	-0.295481	5.335794	-0.753189	-0.047059	1.321881	1
3	3.192094	1.126541	-1.413480	0.085070	0.157979	0.270645	3.582538	-1.183492	-2.159567	0.279097	1
4	-1.175621	-0.873074	0.104280	1.220715	-0.433351	-0.072347	-1.033671	-0.370771	1.230646	-2.161162	0

Data preprocessing and splitting complete!

--- Model Performance Comparison ---

	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Logistic Regression	0.901429	0.938650	0.861972	0.898678	0.950014
1	Decision Tree	0.911429	0.924638	0.898592	0.911429	0.911615
2	Random Forest	0.942857	0.943662	0.943662	0.943662	0.985221
3	Gradient Boosting	0.945714	0.949008	0.943662	0.946328	0.983082

Best Hyperparameters Found: {'learning_rate': 0.05, 'max_depth': 3, 'n_estimators': 100}
Tuned Model Accuracy: 0.9471



Classification Report:

	precision	recall	f1-score	support
0	0.95	0.94	0.95	345
1	0.95	0.95	0.95	355
accuracy			0.95	700
macro avg	0.95	0.95	0.95	700
weighted avg	0.95	0.95	0.95	700

Model and scaler successfully saved as 'best_model.pkl' and 'scaler.pkl'!